

UT-05: INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

¿Qué vamos a ver?

- PROGRAMACIÓN ORIENTADA A OBJETOS
 - CONCEPTOS: CLASE - OBJETO.
 - PROPIEDADES DE LA P.O.O.
- CLASES.
 - CARÁCTERÍSTICAS.
 - CREAR UNA CLASE EN JAVA. FORMATO GENERAL.
 - DEFINICIÓN DE ATRIBUTOS O PROPIEDADES.
 - MODIFICADORES DE ACCESO.
- MÉTODOS.
 - COMUNICACIÓN ENTRE OBJETOS. ENVÍO DE MENSAJES.
 - CONSTRUCTORES.
 - MÉTODOS SETTER Y GETTER
 - EL MÉTODO MAIN.
- OBJETOS.
 - CREAR UN OBJETO.
 - ACCEDER A LOS ATRIBUTOS DE UN OBJETO.
- ATRIBUTOS Y MÉTODOS STATIC.
- CLASES DE INTERÉS

1. PROGRAMACIÓN ORIENTAD A OBJETOS

La programación orientada a objetos permite una representación más directa del modelo del mundo real en el código. El resultado es que se reduce considerablemente la transformación de los requisitos del sistema (definido en términos de usuario) a la especificación del sistema (definido en términos de computador). Es un paradigma de programación que usa objetos en sus interacciones, para diseñar aplicaciones y programas informáticos. Está basada en varias técnicas, incluyendo herencia, cohesión, abstracción, polimorfismo, acoplamiento y encapsulamiento.

Fue a principios de la década de los 90 cuando su uso se popularizó y en la actualidad probablemente sea uno de los paradigmas de programación más utilizados existiendo gran variedad de lenguajes de programación que soportan la orientación a objetos.

2.1. Ventajas de la programación orientada a objetos

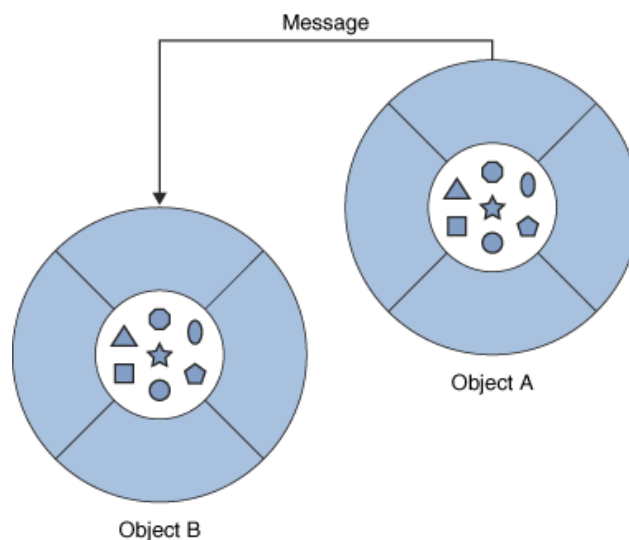
- **Comprensión:** Facilita la comprensión al representar los problemas en términos del mundo real. Es una representación más cercana a nuestra forma de pensar.
- **Modularidad:** Al estar las definiciones de objetos en módulos, hace que las aplicaciones estén mejor organizadas y sean más fáciles de entender.
- **Fácil mantenimiento:** Cualquier modificación en las acciones queda automáticamente reflejada en los datos.
- **Seguridad:** No se pueden modificar los datos de un objeto directamente, sino que se debe hacer mediante las acciones definidas para ese objeto.
- **Reutilización de código:** Facilita la reutilización mediante el uso de bibliotecas de clases.

2.2. Objetos

Un objeto se corresponde con una entidad del mundo real que se caracteriza por un estado (datos) y un comportamiento (operaciones).

- **Estado:** El estado de un objeto viene determinado por los valores que toman sus datos. Los datos se denominan también atributos o propiedades y componen la estructura del objeto. Por ejemplo, un objeto Coche su estado podría estar definido por atributos como Marca, Modelo, Color, etc.
- **Comportamiento:** El comportamiento de un objeto viene determinado por las operaciones (métodos) que se pueden realizar sobre el objeto. Por ejemplo, un objeto Coche su comportamiento podría estar definido por acciones como arrancar, parar, acelerar, etc.

Los objetos se comunican unos con otros llamando a sus métodos. Estos métodos determinan cómo actúan los objetos cuando reciben un mensaje. Un mensaje es la acción que hace un objeto. Y un método especifica cómo se ejecuta un mensaje.



Los objetos son entidades dinámicas, que existen en el tiempo; por ello deben ser creados o instanciados, normalmente a través de otros objetos. Esta operación se hace a través de los métodos especiales llamados constructores, que pueden ser ejecutados implícitamente por el compilador o explícitamente por el programador mediante la invocación a los citados constructores.

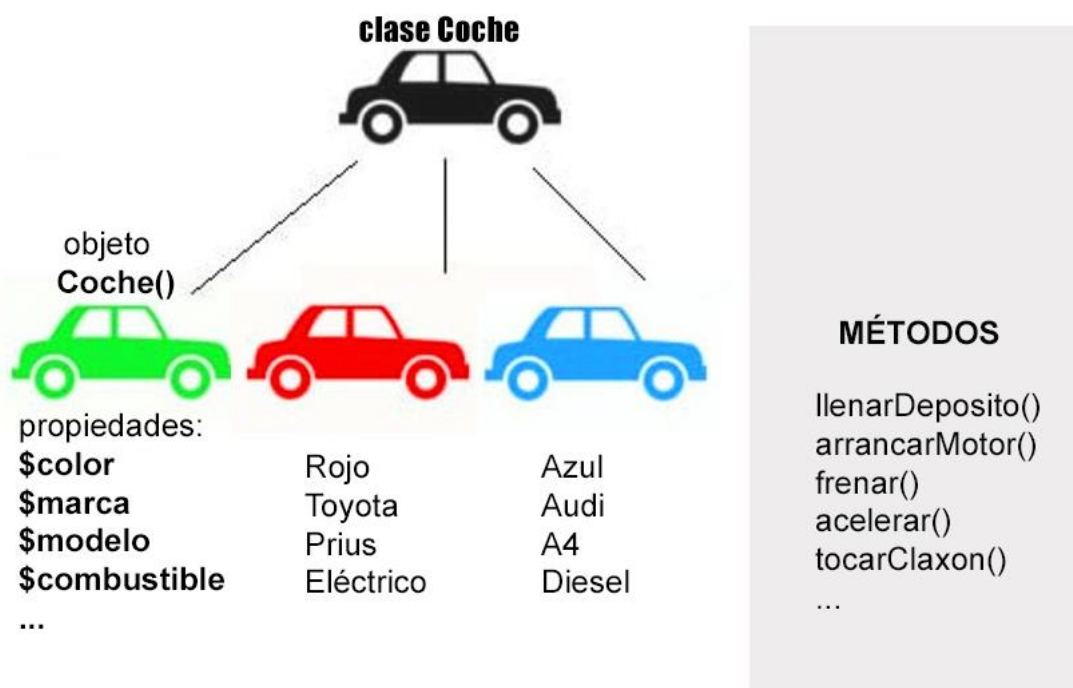
2.3. Clases

Una clase es la caracterización abstracta de un conjunto de objetos y define los datos que se utilizan para representar un objeto y las operaciones que se pueden realizar sobre esos datos. Se pueden definir muchos objetos de la misma clase. Es decir, una clase es la declaración de un tipo objeto.

Las clases son similares a los tipos de datos y equivalen a modelos o plantillas que describen:

- **Los atributos o propiedades comunes** a todos los objetos de la clase.
- **Los métodos o funciones** que son las operaciones que pueden utilizarse para manejar esos objetos.

Cada vez que se construye un objeto a partir de una clase, estamos creando lo que se llama una instancia de esa clase. En general, instancia de una clase y objeto son términos intercambiables. Veamos un ejemplo:



2.4. Propiedades de la programación orientada a objetos

- Abstracción
- Encapsulación
- Herencia
- Polimorfismo

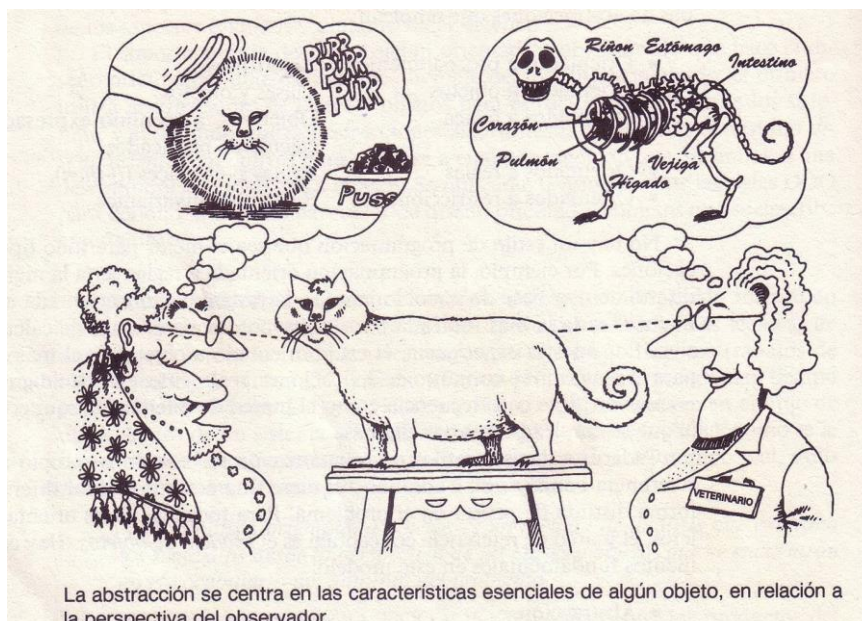
2.4.1. Abstracción

La abstracción es la propiedad que permite representar las características esenciales de un objeto desde un punto de vista determinado y no tener en cuenta las restantes características, reduciendo así la complejidad. Es decir, permite no preocuparse por los detalles no esenciales.

Por ejemplo, diferentes modelos de abstracción del objeto Coche podrían ser:

- Un coche es la combinación de diferentes partes, tales como motor, carrocería, número de puertas, etc.
- Un coche puede clasificarse por el nombre del fabricante, por su categoría (turismo, deportivo, todoterreno ...), por el carburante que utilizan (gasolina, gasoil, híbrido ...).

Por ejemplo, la segunda abstracción vista anteriormente se utilizará siempre que la marca, la categoría o el carburante sean significativos. Por tanto, la abstracción a utilizar dependerá de la naturaleza del problema a resolver.

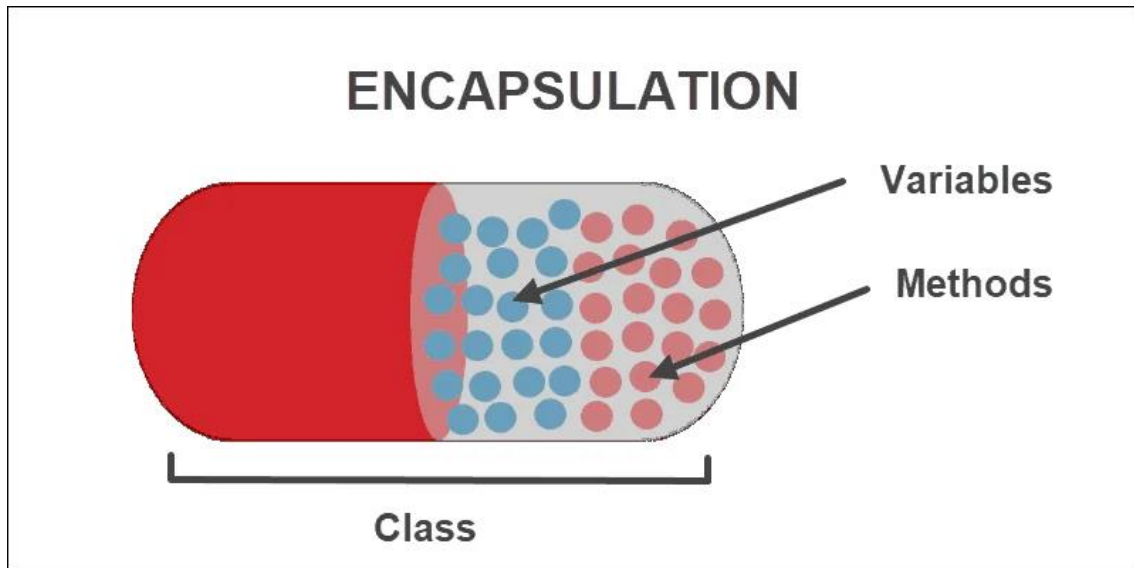


2.4.2. Encapsulación de datos

La encapsulación es la propiedad que permite asegurar que el contenido de la información de un objeto está oculto. La encapsulación (también se conoce como ocultación de la información), en esencia, es el proceso que oculta todos los secretos de un objeto que contribuyen a sus características esenciales.

La encapsulación permite la división de un programa en módulos. Estos módulos se implementan mediante clases, de forma que una clase representa la encapsulación de una abstracción. En la práctica, esto significa que cada clase debe tener dos partes: una interfaz y una implementación. La interfaz de una clase captura sólo su vista externa y la implementación contiene la representación de la abstracción, así como los mecanismos que realizan el comportamiento deseado.



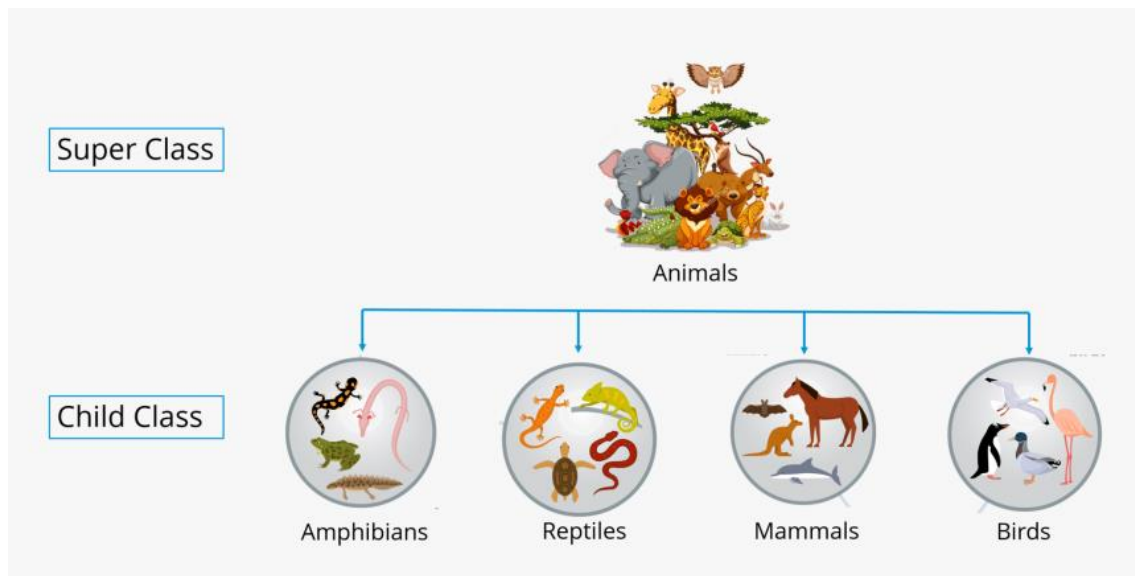


2.4.3. Herencia

La herencia es la propiedad que permite a los objetos ser contruidos a partir de otros objetos. La idea fundamental es permitir crear nuevas clases aprovechando las características (atributos y métodos) de otras clases ya creadas evitando así tener que volver a definir esas características (reutilización).

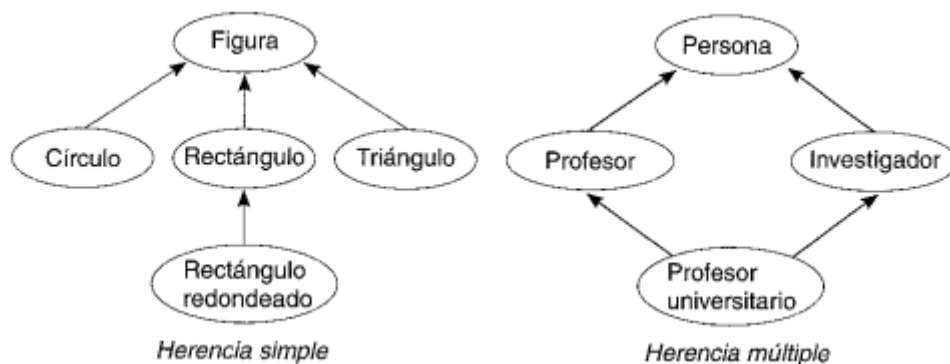
A una clase que hereda de otra se le llama subclase (clase derivada, clase hija) y aquella de la que se hereda es conocida como superclase (clase base, clase padre).

Las clases derivadas heredan el código y datos de su clase base, añadiendo su propio código y datos, incluso puede cambiar aquellos elementos de la clase base que necesita que sean diferentes. Por ejemplo, la clase animal se puede dividir en las subclases anfibios, mamíferos, aves, etc., y la clase vehículo en coches, camiones, etc.



Existen dos tipos de herencia:

- **Herencia simple:** Una subclase puede heredar datos y métodos de una única clase. Java admite solo herencia simple.
- **Herencia múltiple:** Una subclase puede heredar datos y métodos de más de una clase. C++ admite herencia simple y múltiple.



La herencia tiene como ventajas principales la reutilización y compartición de código (evita repetir código ahorrando mucho tiempo), la consistencia de la interfaz (el comportamiento que heredan será el mismo en todos los casos) y la ocultación de información. Pero también tiene inconvenientes como son la velocidad de ejecución (los métodos heredados suelen ser más lentos) y la curva

de aprendizaje (al principio el aprendizaje de la programación orientada a objetos suele ser más lento).

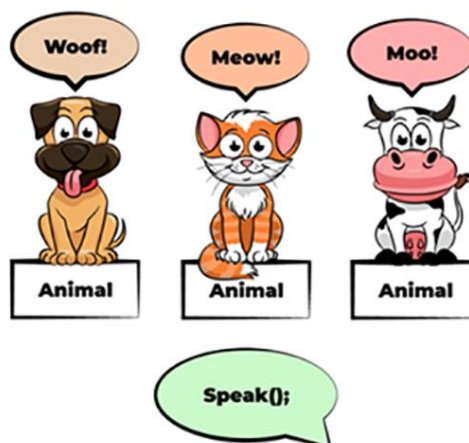
2.4.4. Polimorfismo

El polimorfismo es la propiedad que permite que un operador o un método actúen de modo diferente en función del objeto sobre el que se aplican. Cuando un operador existente en el lenguaje tal como `+`, `=` o `*` se le asigna la posibilidad de operar sobre un nuevo tipo de datos, se dice que está sobrecargado. La sobrecarga es una clase polimorfismo.

En un sentido más general, el polimorfismo supone que un mismo mensaje puede producir acciones totalmente diferentes cuando se reciben por objetos diferentes.

El polimorfismo adquiere su máxima expresión en la herencia de clases, es decir, cuando se obtiene una clase a partir de una clase ya existente.

Por ejemplo, cuando se describe una clase base Mamífero se puede observar que la operación comer es una operación fundamental en su vida, de modo que cada tipo de mamífero (vaca, humano, león) debe poder realizar la operación comer, aunque cada una de las clases derivadas que representan los distintos tipos de mamíferos la realizará de un modo diferente (polimorfismo).



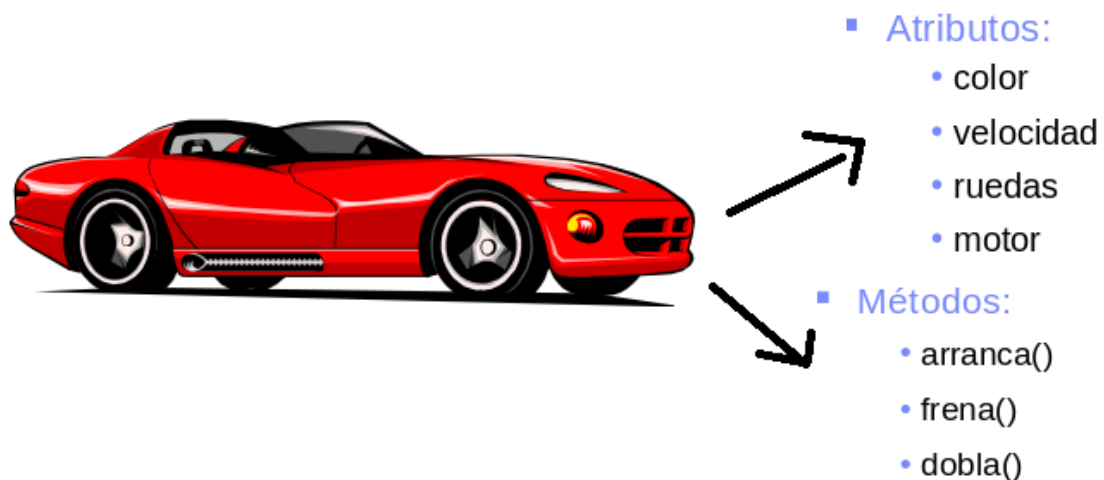
2. CLASES

2.1. Características

Como hemos visto anteriormente una clase se caracteriza por sus atributos (estado) y sus métodos (comportamiento).

Por un lado, los atributos almacenarán datos que consideremos necesarios para la clase y los objetos que se van a crear a partir de ella.

Por otro lado, los métodos serán acciones que la clase podrá llevar a cabo sobre ella o sobre otros elementos.



2.2. Crear un clase en java. Formato general.

Aunque será a partir de la siguiente unidad en la que nos centraremos en la creación de clases, veamos a grandes rasgos cual es estructura general que suelen tener.

Podemos distinguir dos tipos de clases, las principales y las secundarias.

Las principales son las primeras que arranca el programa y contienen el método main.

```
package ut06;

import java.util.Scanner;

public class UT06 {

    private static int numero;

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        String nombre;

        System.out.print("Dame tu nombre: ");
        nombre = scan.nextLine();
        saludar(nombre);
    }

    public static void saludar(String nombre){
        System.out.println("¡Hola " + nombre + "!");
    }

    public class SubUT06 {
        ...
    }
}
```

Las clases principales suelen tener la siguiente estructura:

- Paquete al que pertenecen
- Importaciones de clases de las que se hace uso
- Declaración de la clase
- Variables y/o constantes globales (no aconsejables)
- Método main
- Métodos auxiliares que realizan diversas acciones
- Definición de subclases

Las clases secundarias se utilizan para modularizar el código y hacerlo mucho más legible y organizado. Las vamos a utilizar directamente o a través de creación de objetos para interactuar con la clase principal u otras clases secundarias.

Las clases secundarias suelen tener la siguiente estructura:

```
package ut06;

public class Employee {
    //Atributos
    private String name;
    private int employeeNum;
    private final int COMISIONPERCENTAGE = 10;

    //Método constructor
    public Employee(String name, int employeeNum) {
        this.name = name;
        this.employeeNum = employeeNum;
    }

    //Métodos getters
    public String getName() {
        return name;
    }

    public int getEmployeeNum() {
        return employeeNum;
    }

    //Métodos setters
    public void setName(String name) {
        this.name = name;
    }

    public void setEmployeeNum(int employeeNum) {
        this.employeeNum = employeeNum;
    }

    //Otros métodos
    public double calculateComision(double saleValue) {
        return saleValue * (COMISIONPERCENTAGE/100);
    }
}
```

- Paquete al que pertenecen
- Declaración de la clase
- Atributos
- Método constructor
- Métodos getters

- Métodos setters
- Otros métodos

2.3. Definición de atributos o propiedades

Los atributos son los encargados de conservar el estado de las clases, es decir, almacena información acerca de ella, que tendrá diferentes valores para cada objeto que se cree por medio de esa clase.

	Nombre:	Fernanda	→	str
	Edad:	28	→	int
	Dirección:	Los Jacintos 1329	→	str
	Estatura:	1.65	→	float
	Usa lentes?	Sí	→	bool

	Nombre:	Héctor	→	str
	Edad:	20	→	int
	Dirección:	Maratón 94 Dp. 31	→	str
	Estatura:	1.76	→	float
	Usa lentes?	No	→	bool

Podemos definir los atributos como variables internas de la clase, con la particularidad de que normalmente no se suele permitir el acceso directo a estas variables desde fuera de la clase, sino utilizamos unos métodos especiales tanto para conocer sus valores como para asignarles unos nuevos.

Como variables, o constantes, que son las definiremos de la misma manera que hemos hecho hasta el momento, es decir, el nombre de la variable irá precedido del tipo de dato que va a almacenar.

Para acceder a un atributo de un objeto, en el caso de que nos esté permitido, solo tenemos que poner un punto después de la clase u objeto del tipo clase, según el tipo de clase, y el nombre del atributo. Ejemplo:

```
String valor = objeto.atributo;
```

2.4. Modificadores de acceso

Como hemos comentado en el apartado anterior, normalmente no se permite el acceso a los atributos desde fuera de la clase, para esto utilizamos lo que se denomina modificadores de acceso.

Un modificador de acceso es una palabra clave que antepondremos al tipo de datos del atributo y con el que indicaremos si solo se puede acceder a él desde la propia clase o también se podrá acceder desde fuera.

Por ahora vamos a utilizar tres modificadores de acceso:

- **private:** solo permite que accedamos a la variable, o contante, desde la propia clase que lo ha declarado, ni siquiera desde el propio paquete en el que se encuentra la clase. Es decir, que cuando utilicemos un objeto con un atributo private, no vamos a poder llamar a este como habíamos visto en el apartado anterior. Va a ser solo de uso interno.

```
private String mensaje;
```

- **default:** es el modificador de acceso que se asigna por defecto al atributo si no especificamos ningún otro modificador, es decir, si no ponemos nada delante del tipo de dato del atributo. Este modificador nos permitirá acceder al atributo desde la misma clase o desde alguna otra clase que se encuentre en el mismo paquete que ella.

```
String mensaje;
```

- **public:** normalmente es el menos aconsejable, puesto que permite el acceso al atributo desde cualquier sitio.

```
public String mensaje;
```

Modificador (palabra reservada)	Misma clase	Mismo paquete	Subclase de otro paquete	Resto
<i>private</i>				
(defecto)				
<i>public</i>				

3. MÉTODOS

3.1. Comunicación entre objetos. Envío de mensajes.

La programación orientada a objetos resuelve los problemas mediante la interacción de los objetos. Esta interacción, o comunicación, se lleva a cabo mediante intercambio de mensajes, esos mensajes son recibidos por los métodos y estos son los encargados de dar respuesta o actuar en consecuencia al mensaje. Por tanto, los métodos son los encargados de gestionar el comportamiento de la clase u objeto.

Podríamos decir que los métodos son subprogramas dentro de los programas, al que le asignamos a cada uno de ellos una labor que realizar, y al que llamamos cada vez que necesitemos llevar a cabo dicha labor.

La codificación de los métodos consiste en dos partes, la cabecera y el cuerpo.

- La cabecera está compuesta por un modificador de acceso, el tipo de dato que devuelve, el nombre del método y, entre paréntesis, los argumentos que debemos pasarle con su tipo de dato correspondiente.

```
public double calculateComision(double saleValue)
```

- El cuerpo es el código que va a determinar qué hace el método, y se escribe entre llaves {}. Además, si se trata de una función, es decir, un método que devuelve un resultado, debemos hacer uso de la palabra

reservada “return” cuando queramos finalizar el método y devolver dicho resultado.

```
{  
    return saleValue * (COMISIONPERCENTAGE/100);  
}
```

El método de ejemplo quedaría de la siguiente manera:

```
public double calculateComision(double saleValue) {  
    return saleValue * (COMISIONPERCENTAGE/100);  
}
```

Para llamar a un método utilizamos la clase o un objeto de esa clase, en función del tipo clase, colocamos un punto a continuación, después el nombre del método y entre paréntesis los argumentos que sean necesarios para realizar la llamada.

```
objeto.metodo(argumento1, argumento2);
```

3.2. Constructores

Los constructores son unos métodos especiales de las clases, a través de los cuales creamos los objetos de esas clases. Un constructor tendrá el mismo nombre que la clase y recibirá una serie de parámetros o ninguno, en función de las necesidades de la clase. Normalmente, además de crear en memoria el objeto, se suelen utilizar para inicializar los valores de los atributos de la clase.

```
public class Employee {  
    //Atributos  
    private String name;  
    private int employeeNum;  
    private final int COMISIONPERCENTAGE = 10;  
  
    //Método constructor  
    public Employee(String name, int employeeNum) {  
        this.name = name;  
        this.employeeNum = employeeNum;  
    }  
}
```

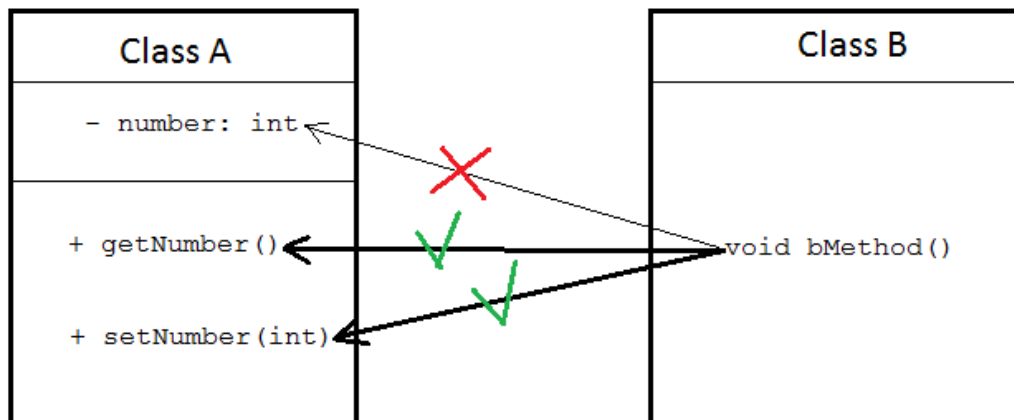
Habitualmente una clase suele tener un único constructor, pero en ocasiones puede tener varios, que se diferencian por el número de parámetros que se le pasan. Incluso hay un tipo especial de clases, sobre las que hablaremos más adelante, que no utilizan constructores, es decir, no se pueden crear objetos a partir de esa clase.

A excepción de la clase principal, los constructores son llamados desde otras clases. Estas llamadas se realizan con una estructura similar a la de la declaración de una variable, ya que los objetos, en realidad, son variables del tipo de la clase, pero utilizando la palabra clave “new”.

```
ClaseAlumno alumno = new ClaseAlumno(argumento1);
```

3.3. Métodos getters y setters

Son otro tipo especial de métodos que utilizamos para acceder a los atributos desde fuera de la clase, en el caso de que sean “private”.



Los métodos getters nos devolverán el valor de un atributo determinado.

Internamente están contruidos como un método normal, pero habitualmente se le da como nombre “getNombreAtributo”, no recibe argumentos y devuelve datos del tipo del atributo.

```
public String getName() {  
    return name;  
}
```

A la hora de llamarlos, se hace igual que un método igual, poniendo “.” Después del objeto, el nombre del método, seguido de “()”.

```
objeto.getNombre();
```

Los métodos setters nos servirán para asignar un valor al atributo correspondiente.

Al igual que los getters, están estructurados como un método normal, pero en este caso no devuelve valor, es decir, pondremos “void” como tipo de dato que devuelve, y como argumento tendremos el valor que quiere asignar con el tipo de dato correspondiente.

```
public void setName(String name) {  
    this.name = name;  
}
```

Para llamar a un método setter, lo haremos igual que otro tipo de método, y le pasaremos como parámetro el valor que queremos asignar al atributo.

```
objeto.setNombre("Juan");
```

Tanto los getters como los setters son herramientas imprescindibles para mantener la encapsulación y el principio de ocultación de la información.

Es importante tener en cuenta que no es necesario crear getters o setters para todos los atributos:

- Podemos tener atributos solo con método getter, es decir, no se le puede modificar el valor que almacenan de forma externa.
- Podemos tener atributos solo con método setter, es decir, solo podemos modificar su valor, pero no leerlo, aunque es menos común.

- Podemos tener atributos sin getter ni setter, ya sea por que es público y no lo necesita, o por que es privado y no queremos que se acceda a él de ninguna manera desde fuera de la clase.

3.4. Método main

El método main es el más importante de un programa. Únicamente se encuentra en la clase principal y siempre es el primer método que se ejecuta, es decir, que es el encargado de llamar al resto cuando sea necesario.

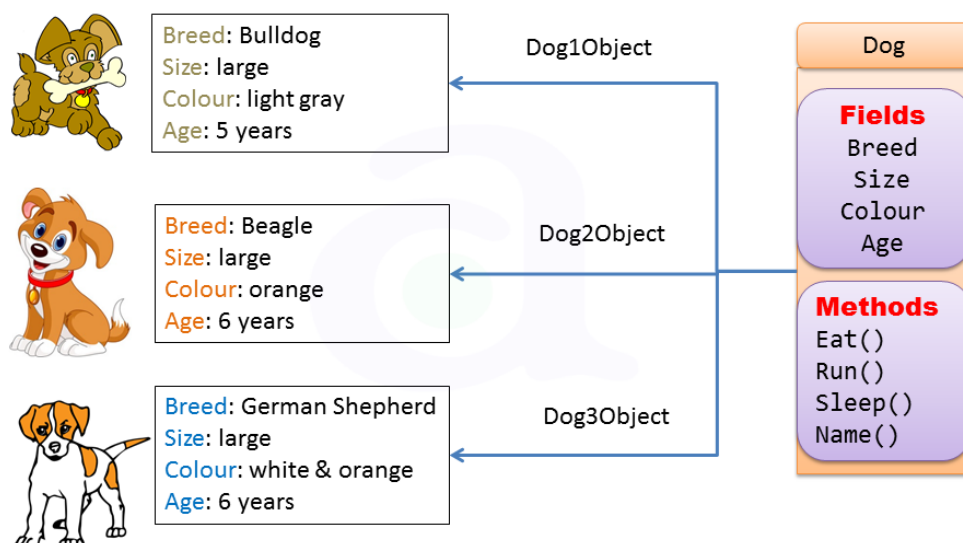
El encabezado del método main siempre es el mismo:

```
public static void main(String[] args)
```

4. OBJETOS

Los objetos los obtenemos gracias la instanciación de las clases, es por ello que también son llamados instancias de las clases. Esto se lleva a cabo por medio de los métodos constructores.

Podríamos decir que las clases son plantillas mediante las cuales creamos objetos similares del mismo tipo.



Cada objeto derivado de una clase contará con los mismos atributos y métodos que esta, pero los valores de los atributos serán distintos.

A continuación, tenemos un ejemplo de cómo crearíamos los tres objetos de clase Dog, y cómo utilizar sus métodos y atributos.

```
Dog Dog1object = new Dog ("Bulldog", "large", "light gray", 5);
Dog Dog2object = new Dog ("Beagle", "large", "orange", 6);
Dog Dog3object = new Dog ("German Shepherd", "large", "white & orange",
6);

Dog1object.eat();
Dog2object.run();
Dog3object.sleep();

Dog2object.setBreed("Chiguagua");

System.out.println("The breed of my dog is " + Dog2object.getBreed());
```

5. ATRIBUTOS Y MÉTODOS ESTÁTICOS

Existen clases que contienen una serie de atributos y métodos especiales a los que denominamos estáticos o de clase.

5.1. Atributos estáticos

Los atributos estáticos son atributos que tienen el mismo valor para todas las instancias, u objetos, de una clase. Este atributo puede ser una constante, o puede ser variable, de tal forma que si se modifica se modificará para todos los objetos de esa clase.

La forma de acceder a un atributo estático, o de clase, es igual que para un atributo de instancia.

Aunque la siguiente unidad aprenderemos a crearlos y el sentido que tienen a la hora de construir nuestras propias clases, a continuación, tenemos un ejemplo de cómo se declara una constante y una variable estática:

```
static final int VALOR_MAXIMO = 100;  
  
static int numEmpleados;
```

5.2. Métodos estáticos

Los métodos estáticos o de clase, son métodos que se caracterizan por que pueden ser utilizados sin necesidad de crear un objeto de la clase, es decir, que simplemente utilizando la clase para llamarlos.

Si recordáis, cuando hablamos de algunas clases de las que llamamos “wrappers”, como Character, ya utilizamos algunos de estos métodos.

```
Character.isLowerCase(letra);
```

En las clases principales todos los métodos secundarios que creemos deberán ser estáticos, debido a la naturaleza de esta clase. Para declarar un método estático solo tenemos que añadir la palabra reservada “static” detrás del modificador de acceso y delante del tipo de datos que va a devolver.

```
public static void saludar(){  
    System.out.println("Hola a todos");  
}  
  
private static int devuelveMayor(int num1, int num2){  
    if(num1 > num2)  
        return num1;  
    else  
        return num2;  
}
```

6. CLASES DE INTERÉS

A continuación tenemos un listado de clases, algunas ya conocidas y otras no, que son de mucha utilidad a la hora de desarrollar aplicaciones. Varias necesitan ser instanciadas para utilizar sus métodos y atributos, mientras que otras no.

- `java.util.Random` (necesita instanciarse)
- `java.util.Scanner` (necesita instanciarse)
- `Byte`
- `Short`
- `Integer`
- `Long`
- `Float`
- `Double`
- `Boolean`
- `Character`
- `String`
- `java.time.LocalDate`
- `java.time.LocalTime`
- `java.time.LocalDateTime`
- `Math`